O Universo Centrado em Serviços do CAP Node.js

Dominando a Dualidade de Provedor e Consumidor



NotebookLM

No CAP, Tudo é um Serviço

Conceito Fundamental

Uma aplicação CAP consiste principalmente nos serviços que ela fornece aos clientes.

Esses serviços, conhecidos como *Provided Services*, são o coração da sua aplicação, expondo seus dados e sua lógica de negócio para o mundo.

Conceito Universal

Ao mesmo tempo, suas aplicações frequentemente precisam consumir outros serviços, conhecidos como *Required Services*.
Pense no banco de dados principal (cds.db), em microsserviços externos ou em *message brokers*. Eles são essenciais para que sua aplicação realize seu trabalho.

No fim das contas, tudo que é ativo no CAP é um serviço, e a classe `cds.Service` é a base para todos eles.

NotebookLM

O Lado Provedor: Definindo o Contrato com CDS

Tudo começa com a modelagem do serviço. Usando a Core Data Services (CDS), você define de forma declarativa as entidades, ações e eventos que seu serviço irá expor. Este é o 'o quê' do seu serviço.

```
// cat-service.cds

using { User, sap.capire.bookshop as my } from '../db/schema';

service CatalogService {
 entity Books {
    key ID : UUID;
    title  : String;
    author : Association to my.Authors;
 }

 action submitOrder ( book: UUID, quantity: Integer );

 event OrderedBook: { book: UUID; quantity: Integer; buyer: User }
}
```

[Annotation: Entidade exposta pela API]

[Annotation: Ação customizada que pode ser chamada]

[Annotation: Evento que o serviço pode emitir]

Esta definição é a única fonte da verdade para a API do seu serviço.

NotebookLM

Da Definição à API: A Mágica do `cds.serve`

Com a definição do serviço em mãos, o framework do CAP assume o controle. Ao iniciar o servidor, o processo de bootstrap automaticamente materializa seus serviços.

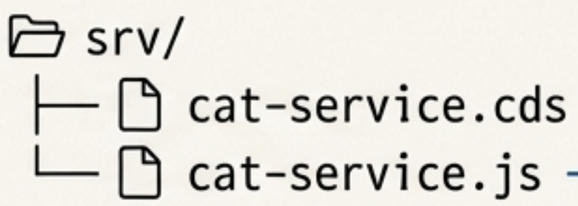


Em essência, o `cds.serve` encontra todas as definições de serviço nos seus modelos, cria uma instância de `cds.ApplicationService` para cada uma e as expõe como endpoints de API, com implementações genéricas para todas as operações CRUD prontas para uso.

NotebookLM

Implementando a Lógica de Negócio Customizada

O CAP fornece CRUD genérico, mas a verdadeira inteligência de negócio está na sua implementação customizada. A maneira mais fácil é criar um arquivo `.js` com o mesmo nome, ao lado do seu arquivo `.cds`.



[Annotation: Implementação reside aqui]

```
// cat-service.js
const cds = require('@sap/cds');

class CatalogService extends cds.ApplicationService {
  init() {
    const { Books, Authors } = this.entities;

    // Adiciona validação ANTES de uma leitura
    this.before('READ', Authors, req => { /* ... */ });

    // Enriquece dados DEPOIS de uma leitura
    this.after('READ', Books, books => { /* ... */ });

    // Implementa a ação customizada 'submitOrder'
    this.on('submitOrder', req => { /* ... */ });

    return super.init();
  }
}

module.exports = CatalogService;
```

NotebookLM

A Outra Face: Configurando Serviços Requeridos

Nenhuma aplicação é uma ilha. Para funcionar, ela precisa interagir com outros serviços. O exemplo mais proeminente é o banco de dados (`cds.db`), mas também inclui outros microsserviços ou serviços de plataforma.

Configuração em `package.json`

A configuração para serviços requeridos atua como um 'soquete', definindo o que você precisa. As credenciais e detalhes de conexão são injetados posteriormente.

```
"cds": {
    "requires": {
        "db": {
            "kind": "sqlite",
            "credentials": { "url": "db.sqlite" }
        },
        "ReviewsService": {
            "kind": "odata",
            "model": "@capire/reviews"
        }
    }
}
```

Conectando via Código

Uma vez configurado, você pode se conectar ao serviço em seu código para obter um objeto cliente para interagir com ele.

```
const cds = require('@sap/cds');

// Conecta-se ao serviço de banco de dados
const db = await cds.connect.to('db');

// Conecta-se a um serviço OData externo
const ReviewsService = await cds.connect.to('ReviewsService');
```

NotebookLM

Interagindo com Serviços: Um Arsenal de APIs

Com a instância do serviço em mãos (via `cds.connect.to`), você tem um conjunto poderoso e flexível de APIs para enviar requisições, executar queries e emitir eventos.

API estilo REST

```
await srv.read('GET','/Books/206')
```

API Tipada (Ações)

```
await srv.submitOrder({
  book:206, quantity:1
})
```

API estilo Query (CQN)

```
await srv.run(
  SELECT.from(Books,201)
)
```

API de Conveniência CRUD

```
await srv.read(Books,201)
```

Emissão de Eventos Assíncronos

```
await srv.emit ('OrderedBook', { book: 206, ... })
```

Prefira APIs Agnósticas de Plataforma

APIs baseadas em `cds.ql` como `srv.run()` são amplamente agnósticas. O CAP pode traduzi-las para chamadas de API locais, OData, GraphQL ou SQL puro. Em contraste, `srv.send()` pode se tornar específico do protocolo (ex: usando query options `$filter` do OData).

NotebookLM

O DNA em Comum: A Classe `cds.Service`

“Tudo que é ativo no CAP é um serviço, e a classe `cds.Service` é a classe base para todos.”

Seja uma instância criada por `cds.serve()` para expor uma API, ou um proxy de cliente obtido por `cds.connect()`, ambos são, em sua essência, instâncias de `cds.Service`. Eles reagem a eventos através da execução de handlers registrados.

```
// Essência de como os serviços funcionam:
const srv = (new cds.Service)
  .on('READ','Books', req => console.log(req.event, req.entity))
  .on('foo', req => console.log(req.event, req.data));

// Consumo:
await srv.read('Books');        //> READ Service.Books
await srv.send('foo',{bar:1}); //> foo {bar:1}
```

Na prática, você raramente cria instâncias desta forma. Em vez disso, você as obtém através do bootstrap do CAP.

NotebookLM

Anatomia de uma Instância de Serviço

Toda instância de `cds.Service` carrega informações essenciais sobre seu nome, modelo e as definições que expõe.

.name: string



O nome do serviço. Se de `cds.serve()`, é o nome qualificado da definição CDS. Se de `cds.connect()`, é o nome usado na busca (ex: ‘audit-log’).

```
const srv = await cds.connect.to('audit-log');
srv.name //> 'audit-log'
```

.model: LinkedCSN



O modelo CDS a partir do qual o serviço foi construído, dando acesso a toda a definição.

.entities, .events, .operations: Iterable



Propriedades convenientes para acessar as definições de CSN das entidades, eventos e operações expostas.

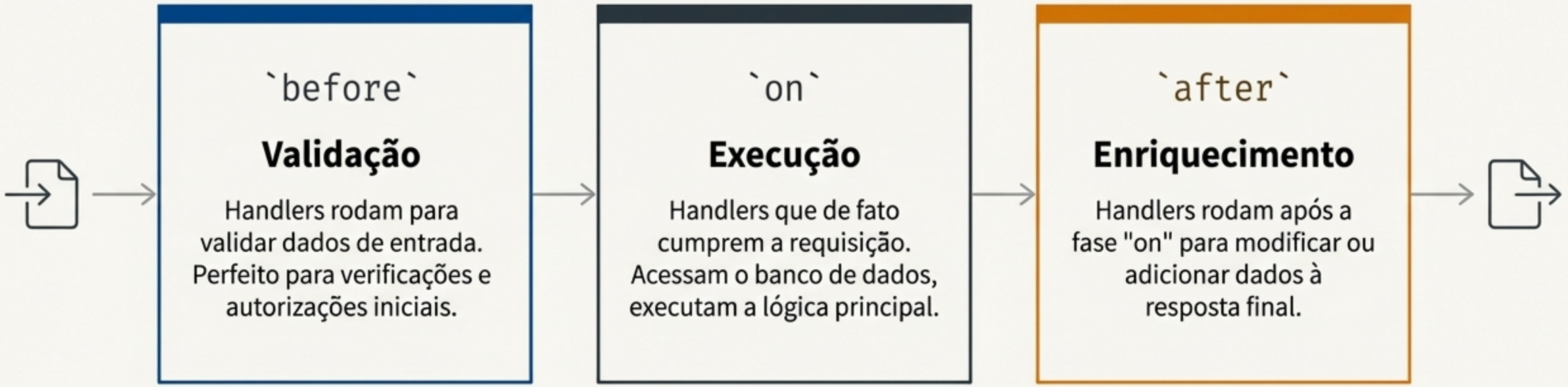
```
// 'this' é uma instância de cds.Service
let { Books, Authors } = this.entities;

for (let entityDef of this.entities) {
  //... entityDef é uma definição de CSN
}
```

NotebookLM

O Ciclo de Vida de uma Requisição: `before`, `on`, `after`

O processamento de qualquer requisição em um serviço CAP segue um ciclo de vida de três fases bem definidas. Entender essas fases é a chave para implementar lógicas robustas e organizadas.



Cada fase é executada em sequência. Se ocorrer um erro em uma fase, as fases subsequentes não são executadas.

NotebookLM

Registrando Handlers para Cada Fase do Ciclo

Você registra handlers em seu arquivo de implementação de serviço, geralmente dentro do método `init()`, usando os métodos `.before()`, `.on()`, e `.after()`.

```
class BooksService extends cds.ApplicationService {
  init() {
    // --- Fase 1: BEFORE ---
    this.before(['CREATE','UPDATE'], 'Books', req => {
      // Valida o estoque recebido
      if (req.data.stock < 0) req.error(400, 'Estoque não pode ser negativo');
    });

    // --- Fase 2: ON (Apenas para ações customizadas ou override) ---
    this.on('submitOrder', async req => {
      // Lógica principal da ação
      const { book, quantity } = req.data;
      await UPDATE(Books, book).set('stock -=', quantity);
      // ...
    });

    // --- Fase 3: AFTER ---
    this.after('READ', 'Books', books => {
      // Enriquece os livros com um campo de desconto
      books.forEach(b => { if (b.stock > 111) b.discount = '11%' });
    });
    return super.init();
  }
}
```

NotebookLM

Dominando Handlers: Interceptores vs. Ouvintes

`srv.on(event, entity, (req, next) => { ... })`

Requisições Síncronas (Interceptores)

Para requisições (CREATE, READ, UPDATE, DELETE, ações), os handlers `.on` formam uma 'pilha'. Cada handler pode processar a requisição e decidir se deve passar o controle para o próximo na cadeia chamando `next()`, ou encerrar o processamento.

CASO DE USO: AUTORIZAÇÃO

```
// Autorização como primeiro handler
this.on('*', function authorize (req, next) {
  if (!req.user.is('authenticated-user')) {
    return req.reject(403, 'FORBIDDEN');
  }
  return next(); // Passa para o próximo handler
});
```

`srv.on(event, (msg) => { ... })`

Eventos Assíncronos (Ouvintes)

Para eventos emitidos via `srv.emit()`, os handlers `.on` agem como 'ouvintes'. **Todos** os handlers registrados para o evento são executados concorrentemente. Não há `next()`.

CASO DE USO: REAÇÕES A EVENTOS DE NEGÓCIO

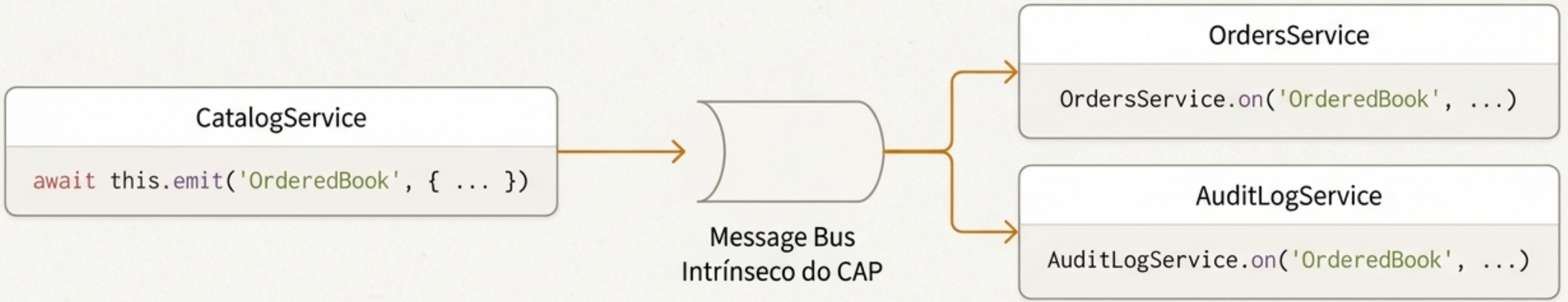
```
// Handler 1: cria um Pedido
BooksService.on('BooksOrdered', async msg => {
  /* ... cria pedido ... */ });

// Handler 2: loga para auditoria
BooksService.on('BooksOrdered', async msg => {
  /* ... loga auditoria ... */ });
```

NotebookLM

Comunicação Assíncrona com `srv.emit()`

Serviços podem emitir mensagens de eventos para notificar outros sobre ocorrências de negócio. Isso permite um design desacoplado, onde os serviços não precisam se conhecer diretamente.



⚠ Atenção ao `await`!

O método `srv.emit()` é assíncrono. É crucial sempre chamá-lo com `await` para garantir o manejo correto das Promises. Não fazer isso pode levar a estados de transação inválidos e deadlocks.

NotebookLM

Síntese da Dualidade: Padrões Chave para o Sucesso

Dominar os serviços core do CAP Node.js é entender como uma aplicação desempenha seus dois papéis fundamentais: o de Provedor e o de Consumidor. Ambos são lados da mesma moeda, unificados pela classe `cds.Service`.

1 **DEFINIR o Contrato (O Quê)**
Use arquivos `.cds` para modelar serviços, entidades e ações.

2 **IMPLEMENTAR a Lógica (Como)**
Crie arquivos `.js` que estendem `cds.ApplicationService`.
Use o ciclo `before`, `on`, `after` para estruturar seu código.

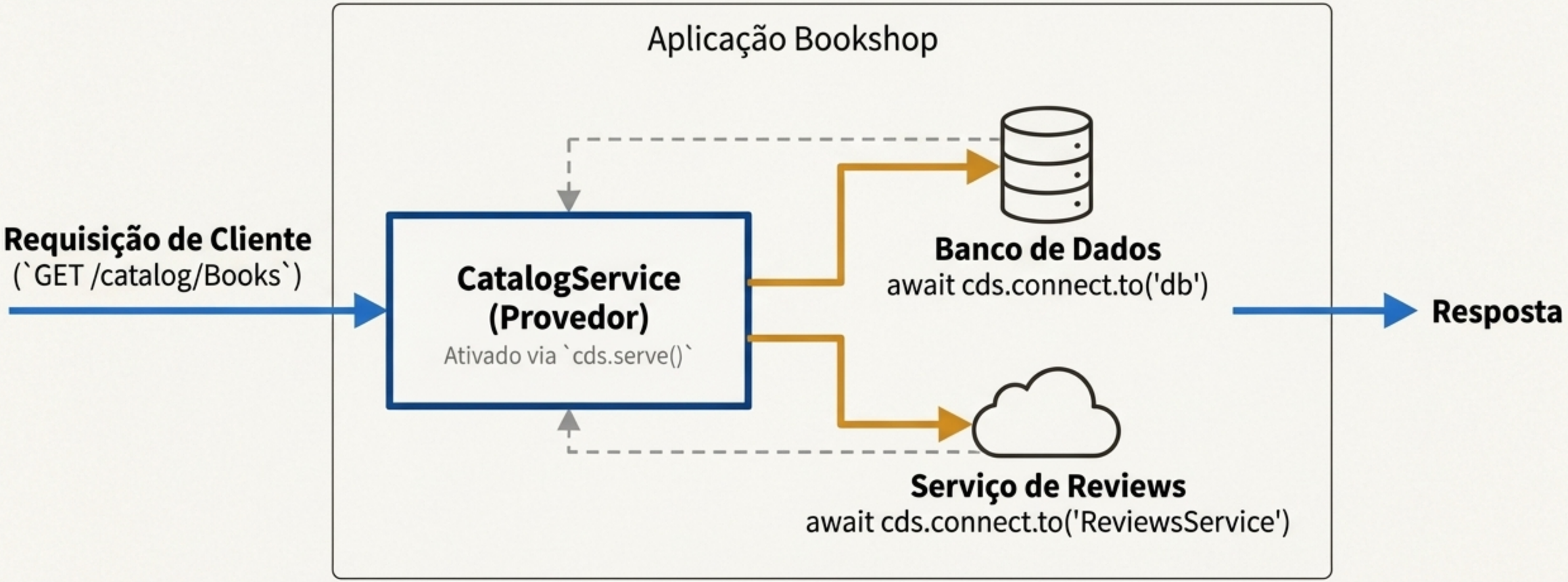
3 **CONFIGURAR Dependências**
Declare serviços requeridos na seção `cds.requires` do seu `package.json`.

4 **CONECTAR e CONSUMIR**
Obtenha instâncias de cliente com `await cds.connect.to('ServiceName')`.
Prefira APIs agnósticas de plataforma como `srv.run(CQN)` para interagir.

5 **COMUNICAR de Forma Assíncrona**
Use `srv.emit()` para publicar eventos de negócio e `srv.on()` para reagir a eles.

NotebookLM

O Universo em Ação: Uma Visão Integrada



Ao ver uma aplicação CAP como um ecossistema de serviços que provêm e consomem capacidades, você pode construir sistemas complexos de forma modular, robusta e escalável.

NotebookLM